Java 1.0–1.4

The foundations

Java 1.0 fixed the shape of the language: classes, interfaces, exceptions, a virtual machine with garbage collection, threads. The 1.x releases filled in the library: reflection, collections, a real JIT, NIO. Everything later stands on this floor.

Classes, interfaces, exceptions → **Bytecode on a VM with GC** → **Look inside: reflection, inner classes** → **A real collections framework** → **HotSpot, regex, NIO**

Java 1.0 Jan 1996
Object model · JVM and GC · Threads · Applets

Java 1.1 Feb 1997
Inner classes · Reflection · JDBC · Serialization

Java 1.2 "Playground" Dec 1998
Collections · Swing · "Java 2"

Java 1.3 "Kestrel" May 2000
HotSpot by default · Dynamic proxies · JNDI

Java 1.4 "Merlin" Feb 2002
assert · Regex · NIO · Logging

1 The object model

What a Java program is made of, and how it fails.

1 Classes and inheritance

Java 1.0 · JLS ch. 8

One superclass, Object at the root

A class is a blueprint: fields hold the state, methods hold the behaviour, constructors set up a new object. `new Circle(2.0)` creates an instance on the heap and hands back a *reference* to it. Variables hold references, never the objects themselves, which is why two variables can point at the same object and why `==` asks "same object?" rather than "same value?". A class can `extend` exactly one other class and inherits its fields and methods; every class without an explicit parent extends `java.lang.Object`, so every object has `equals`, `hashCode`, `toString` and `getClass`.

```
abstract class Shape {
    abstract double area();   // virtual
}
final class Circle extends Shape {
    private final double r;
    Circle(double r) { this.r = r; }
    double area() { return Math.PI * r * r; }
}
```

USE IT WHEN Things that carry state and behaviour together, and one type that is a genuine specialisation of another (a `Circle` *is a* `Shape`), not merely something similar to it.

DON'T Don't inherit just to reuse code. `java.util.Stack` extends `Vector` to reuse its storage, so every `Vector` method became part of `Stack`'s public API, including `insertElementAt`, which lets a caller insert into the middle of a "stack". It can never be removed without breaking programs. Put the reused class in a field instead (composition).

DO Make classes `final` unless you designed them to be extended, make every field `private`, and override `equals` and `hashCode` together or not at all.

REMEMBER *Is-a → extends. Has-a → a field. If you hesitate, it is has-a.*

IN DEPTH

Inheritance and overriding. When `Circle extends Shape`, a `Circle` can be used wherever a `Shape` is expected. If the subclass defines a method with the same signature as its parent, it *overrides* it, and a call such as `s.area()` runs the version belonging to the object's actual class, decided at run time. This is called dynamic dispatch, and in Java every non-private, non-static, non-final method works this way by default (in C++ you must write `virtual`). It is what lets a `List<Shape>` hold circles and squares and compute each area correctly without a single `if`.

The modifiers. `abstract` on a class means "cannot be instantiated, only extended"; on a method it means "no body here, subclasses must provide one". `final` on a class forbids subclassing, on a method forbids overriding, on a field or variable forbids reassignment after initialisation. `static` means "belongs to the class, not to any instance": one copy shared by all objects.

Access levels. `private` : visible only inside the class. No modifier (package-private): visible to classes in the same package. `protected` : same package plus subclasses anywhere. `public` : everyone. Packages use reverse domain names (`com.ballys.payments`) so that two companies' classes never collide.

Why the Stack example matters. Inheritance exposes the parent's whole public API as part of the child's. If the parent later gains a method, the child gains it too, whether it makes sense or not. Composition (keeping a `Vector` in a private field and exposing only `push` , `pop` and `peek`) lets you choose exactly what callers may do. The JDK itself now tells you to use `ArrayDeque` instead of `Stack` .

WHY IT EXISTS

James Gosling wanted the productivity of C++ objects without the things that made C++ programs crash: no multiple inheritance of implementation (so there is never an argument about which parent's method runs), no pointer arithmetic, no manual memory management. Single inheritance plus interfaces was the compromise.

TRY IT YOURSELF

Open `jshell` (ships with every JDK since 9) and paste the lines below one at a time. The first block shows dynamic dispatch through a `Shape` reference; the second shows the broken stack contract.

```
jshell> abstract class Shape { abstract double area(); }
jshell> class Circle extends Shape { double area() { return 3.14; } }
jshell> Shape s = new Circle(); s.area()   // 3.14, Circle's version

jshell> var st = new java.util.Stack<Integer>();
jshell> st.push(1); st.push(2); st.push(3);
jshell> st.insertElementAt(99, 0)         // a Vector method
jshell> st                                // [99, 1, 2, 3]: not a stack any more
```

REFERENCES

Java Language Specification, chapter 8, Classes · `docs.oracle.com/javase/specs/jls/se21/html/jls-8.html`

Java Tutorials: Classes and Objects · `docs.oracle.com/javase/tutorial/java/javaOO/index.html`

java.util.Stack API note recommending Deque · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Stack.html`

Effective Java, 3rd edition, Item 18: favor composition over inheritance (book)

2 Interfaces

Java 1.0 · JLS ch. 9

A contract with no implementation

An interface is a list of method signatures with no bodies (until Java 8 added default methods). A class that `implements` an interface promises to provide every method on the list, and from then on it can be used wherever that interface type is expected. A class may implement any number of interfaces, so Java gets multiple inheritance of *type* ("a Invoice is a Payable and a Runnable") without the ambiguity of multiple inheritance of *implementation*: there is no code in the interface, so there is nothing to clash.

```
interface Payable {
    double amount();
}
class Invoice implements Payable, Runnable {
    public double amount() { return total; }
    public void run() { send(this); }
}
Payable p = new Invoice();   // depend on this
```

USE IT WHEN Whenever a caller should depend on what something can do, not on which class it is. Parameters, fields and return types should name the interface, so the implementation can be swapped (a fake in a test, a different vendor in production).

DO Program to the interface: `List<String> xs = new ArrayList<>()` . Keep interfaces small; one or two methods is normal, not a smell.

DON'T Don't use an interface as a bag of constants (`interface Consts { int MAX = 9; }`): it leaks the constants into every implementing class's namespace. And don't add a method to an interface you have published: every existing implementor stops compiling (default methods in Java 8 exist precisely to soften this).

REMEMBER *Interface = can-do. Class = is-a. Runnable, Comparable, Closeable: all can-dos.*

IN DEPTH

Interface as a type. `Payable p = new Invoice()` declares `p` with the interface type. Through `p` you can only call `amount()` , nothing else that `Invoice` happens to have. That restriction is the point: the code using `p` cannot accidentally depend on `Invoice` details, so tomorrow `p` can hold a `CreditNote` without any change.

JDBC is the textbook case. `java.sql.Connection` , `Statement` and `ResultSet` are interfaces in the JDK. Oracle, PostgreSQL and MySQL each ship a driver JAR with their own classes implementing them. Your code never names a vendor class; you ask `DriverManager` for a `Connection` and work through the interfaces. Change the JDBC URL and the driver JAR, and the same program talks to a different database.

Marker interfaces have no methods at all: `Serializable` and `Cloneable` only tag a class so the runtime can treat it specially. Today you would use an annotation for that.

Later evolution. Java 5 made interfaces generic (`Comparable<T>`). Java 8 added default and static methods with bodies, and single-method interfaces became the target type of lambdas (chapter 4). Java 9 added private methods, Java 17 sealed interfaces that list their permitted implementors (chapter 8).

WHY IT EXISTS

Gosling borrowed the idea from Objective-C protocols: a way to say "this object can do X" independently of its position in the class tree. It is what makes libraries pluggable; without interfaces, every framework would have to know your concrete classes.

TRY IT YOURSELF

In `jshell` , define one interface and two implementations, then write a method that accepts the interface. Swap the implementation without touching the method.

```
jshell> interface Greeter { String greet(String n); }
jshell> class Polite implements Greeter { public String greet(String n) { return "Good day, " + n; } }
jshell> class Casual implements Greeter { public String greet(String n) { return "Hey " + n; } }
jshell> String welcome(Greeter g) { return g.greet("Kamal") + "!"; }
jshell> welcome(new Polite())   // "Good day, Kamal!"
jshell> welcome(new Casual())   // "Hey Kamal!"  same method, other behaviour
```

REFERENCES

Java Language Specification, chapter 9, Interfaces · `docs.oracle.com/javase/specs/jls/se21/html/jls-9.html`

Java Tutorials: Interfaces · `docs.oracle.com/javase/tutorial/java/IandI/createinterface.html`

Java Tutorials: JDBC overview (interfaces implemented by drivers) · `docs.oracle.com/javase/tutorial/jdbc/overview/index.html`

3 Exceptions

Java 1.0 · JLS ch. 11 · cause chaining 1.4

Failure modes in the signature

When something goes wrong, a method `throw` s an exception object instead of returning. The JVM then unwinds the call stack, abandoning each method in turn, until it finds a `catch` block whose type matches. If none does, the thread dies and prints the stack trace. The exception types form a tree under `Throwable` : `Error` for JVM-level trouble you should not catch (out of memory, stack overflow), `RuntimeException` for programming mistakes (`NullPointerException` , `IndexOutOfBoundsException`), and every other `Exception` subclass, the *checked* exceptions, which a method must either handle or announce in its `throws` clause.

```
void charge(Card c) throws PaymentException {
    try { gateway.send(c); }
    catch (IOException e) {
        throw new PaymentException("send", e);
    } finally {
        gateway.close();     // every exit path
    }
}
```

USE IT WHEN Checked exceptions for conditions a sensible caller can recover from (file missing, network down, invalid input from a user). Unchecked for bugs the caller cannot fix at run time (null where it should not be, bad index, violated precondition). `finally` for releasing whatever you acquired, whichever way the block exits.

DO When you wrap a low-level exception in your own, always pass the original as the *cause* (second constructor argument), and catch the most specific type you can actually handle, nothing broader.

DON'T Don't write `catch (Exception e) {}` and carry on (the failure vanishes), don't both log and rethrow the same exception (it appears twice in the logs), and never `return` from inside `finally` : it silently throws away any exception that was in flight.

REMEMBER *Checked = the caller's problem, in the signature. Unchecked = the programmer's problem, in the stack trace.*

IN DEPTH

Checked versus unchecked, concretely. If `Files.readString(path)` is called in a method that neither catches `IOException` nor declares `throws IOException` , the program does not compile. That is a checked exception: the compiler forces the caller to decide. `list.get(99)` on a three-element list throws `IndexOutOfBoundsException` , a `RuntimeException` , and the compiler says nothing: it is a bug, and the fix is in the code, not in a catch block.

What finally guarantees. The `finally` block runs whether the `try` block completed normally, returned early, or threw. Before Java 7 it was the only way to be sure a file or connection was closed. Java 7's try-with-resources (chapter 3) does the same for anything `AutoCloseable` in one line and handles the awkward case where both the body and `close()` throw.

Cause chaining (1.4). A domain-level exception such as `PaymentException` is more useful to the caller than a raw `IOException` , but wrapping used to lose the original stack trace. Since 1.4 every `Throwable` can carry a `cause` ; the stack trace then prints the chain as `Caused by: java.io.IOException ...` , so you see both the business meaning and the technical root.

Reading a stack trace. The first line is the exception type and message. Each `at` line is one frame, innermost first: the top frame is where it was thrown, the first frame in *your* package is usually where to look. `Caused by` sections repeat this for each wrapped cause; the last one is the real origin.

WHY IT EXISTS

C signals failure with return codes that callers forget to check, so errors propagate silently as corrupted state. Exceptions make a failure impossible to ignore by accident, keep the error-handling code out of the happy path, and carry a stack trace to the point of failure.

TRY IT YOURSELF

Create a file `Ex.java` with the code below and compile it. First you will get a compile error (checked exception not handled); add `throws Exception` to `main`, run it, and read the `Caused by` line in the trace.

```
public class Ex {
    static void load() throws java.io.IOException {
        throw new java.io.IOException("disk gone");
    }
    public static void main(String[] a) {   // add: throws Exception
        try { load(); }
        catch (java.io.IOException e) {
            throw new IllegalStateException("startup failed", e);
        }
    }
}
// $ java Ex.java    -> ... IllegalStateException: startup failed
//                      Caused by: java.io.IOException: disk gone
```

REFERENCES

Java Language Specification, chapter 11, Exceptions · `docs.oracle.com/javase/specs/jls/se21/html/jls-11.html`

Java Tutorials: Exceptions · `docs.oracle.com/javase/tutorial/essential/exceptions/index.html`

Throwable API (chained exceptions, getCause) · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Throwable.html`

2 The machine underneath

What runs your bytecode, and what it promises.

4 JVM and bytecode

Java 1.0 · JVM specification

Write once, run anywhere (mostly)

Running a Java program is a two-step process. First `javac` compiles your `.java` source into a `.class` file. That file does not contain instructions for your Intel or Apple CPU; it contains *bytecode*: instructions for an imaginary, standardised computer called the Java Virtual Machine. Second, the `java` command starts a real JVM program on your machine, which loads the class file, checks it, and executes the bytecode, translating it into instructions your actual CPU understands as it goes. Because the class file only ever targets the imaginary machine, the same file runs unchanged on Windows, Linux and macOS, on Intel or ARM, as long as a JVM exists there. That is what "write once, run anywhere" means.

```
$ javac Hello.java        # -> Hello.class
$ javap -c Hello          # read the bytecode
  0: getstatic     #7   // System.out
  3: ldc           #13  // "hi"
  5: invokevirtual #15  // println
  8: return
$ java Hello              # any OS, any JVM
```

USE IT WHEN Understanding stack traces, classloading errors (`ClassNotFoundException` versus `NoClassDefFoundError`), and why a JAR compiled with Java 21 refuses to start on Java 17 (`UnsupportedClassVersionError`: the class file says which JVM version it needs).

DON'T Don't assume "run anywhere" covers behaviour as well as bytecode: file path separators, line endings, the default character set (UTF-8 everywhere only since Java 18) and the locale still differ between platforms.

DO Spend ten minutes with `javap -c` on a small class of your own (steps below). Seeing that a string `+` becomes a builder call, that an `int` in a `List` becomes `Integer.valueOf`, and that `synchronized` becomes `monitorenter` / `monitorexit` instructions makes those three topics concrete for good.

REMEMBER *Source → bytecode → JIT. The class file is the contract; the JVM is the platform.*

IN DEPTH

What is in a class file. A constant pool (all the strings, numbers, class and method names the code refers to), the list of fields and methods, and for each method its bytecode: a sequence of one-byte opcodes such as `getstatic` (read a static field), `ldc` (push a constant), `invokevirtual` (call a method) and `return`. The JVM is a *stack machine*: instructions push values onto an operand stack and pop them again, so `a + b` becomes "push a, push b, `iadd`". The format is defined in the JVM Specification, chapter 4, and it is the real contract of the platform: Kotlin, Scala, Groovy and Clojure compilers produce the same kind of class files, which is why they can use every Java library.

Loading, linking, verifying. When a class is first needed, a *classloader* finds its bytes (on the classpath, in a JAR, over the network in the applet days) and the JVM *verifies* them before running anything: every instruction must be type-correct, the stack must never underflow, no instruction may forge a reference to memory it should not see. Verification is why a corrupted or malicious class file fails at load time instead of corrupting the heap. It still runs today on every class you load.

Applets and the sandbox, explained. An *applet* was a small Java program embedded in a web page; the browser downloaded its class files and ran them inside the browser's JVM. Running code from an unknown website on your machine is dangerous, so Java 1.0 came with a *sandbox*: the verifier guaranteed the bytecode was well-formed, and a `SecurityManager` plus the classloader refused applet code access to the local file system, to other websites, and to native libraries. This was the first mainstream "safe to run downloaded code" story, and it is a large part of why Java became popular in 1996. Browsers dropped plug-in support around 2015, the Applet API was deprecated in Java 9 and removed in Java 26 (JEP 504); the Security Manager was disabled for good in Java 24 (JEP 486).

JNI, the escape hatch. The Java Native Interface (1.1) lets Java code call functions written in C or C++ (and the reverse), through methods declared `native` and a shared library loaded with `System.loadLibrary`. It exists for the cases the sandbox and the standard library cannot cover: talking to an operating-system API, reusing an existing native library, or squeezing performance. The price is that native code steps outside every guarantee the JVM gives (no verification, no garbage collection of what it allocates, a crash takes the whole process down). Java 22's Foreign Function & Memory API (JEP 454, chapter 11) is the modern replacement; Java 24 started warning when a library uses JNI without permission (JEP 472).

Three errors you will meet. `ClassNotFoundException`: code asked for a class by name (`Class.forName`, a plug-in, a JDBC driver) and the classloader could not find it: usually a missing JAR. `NoClassDefFoundError`: the class was present at compile time but is missing or failed to initialise at run time, often a dependency version mismatch. `UnsupportedClassVersionError`: the class file's major version (52 = Java 8, 61 = 17, 65 = 21, 69 = 25) is newer than the running JVM; compile with `--release 17` or upgrade the runtime.

WHY IT EXISTS

In 1995 Sun was targeting set-top boxes and then the web, with no idea which CPUs would be out there. Compiling to a virtual machine meant one compiled artifact for every device, and verification made it safe to run code that arrived over the network. Smalltalk had shown that a VM with garbage collection could be practical; Java made it mainstream.

TRY IT YOURSELF

How to learn `javap -c` in ten minutes. Save the file below, compile it, and run `javap -c -p Peek` (`-c` prints the bytecode, `-p` includes private members). Then look for three things: in `concat`, the `invokedynamic ... makeConcatWithConstants` instruction is the `+` (on Java 8 you would see a `StringBuilder` instead); in `box`, the `Integer.valueOf` call that the compiler inserted to put an `int` into a `List`, and `intValue` to take it out; in `locked`, the `monitorenter` and `monitorexit` instructions that are all `synchronized` really is, with a second `monitorexit` in an exception handler so the lock is released if the body throws.

```
import java.util.*;
public class Peek {
    String concat(String a, int n) { return a + "-" + n; }
    int box(List<Integer> xs) { xs.add(7); return xs.get(0); }
    int n;
    void locked() { synchronized (this) { n++; } }
}
// $ javac Peek.java && javap -c -p Peek
```

REFERENCES

The Java Virtual Machine Specification (Java SE 21) · `docs.oracle.com/javase/specs/jvms/se21/html/index.html`

JVMS chapter 4: the class file format · `docs.oracle.com/javase/specs/jvms/se21/html/jvms-4.html`

JVMS section 4.10: verification of class files · `docs.oracle.com/javase/specs/jvms/se21/html/jvms-4.html#jvms-4.10`

javap command reference · `docs.oracle.com/en/java/javase/21/docs/specs/man/javap.html`

Java Native Interface specification · `docs.oracle.com/en/java/javase/21/docs/specs/jni/index.html`

JEP 504: Remove the Applet API (Java 26) · `openjdk.org/jeps/504`

JEP 486: Permanently disable the Security Manager (Java 24) · `openjdk.org/jeps/486`

JEP 454: Foreign Function & Memory API (Java 22) · `openjdk.org/jeps/454`

5 Garbage collection

No free(), no dangling pointers

In C you ask for memory with `malloc` and must give it back with `free`. Forget to free and the program slowly eats memory; free too early and some other code is still using it, which corrupts data or opens a security hole. Java removes the second half of the job: you create objects with `new` and never free them. The JVM's garbage collector (GC) periodically works out which objects the program can no longer reach and reclaims their memory automatically. The cost is that the collector needs CPU time and, with older collectors, pauses the whole program while it works.

```
// Leaks that GC cannot fix:
static Map<Key, Big> cache = new HashMap<>();
cache.put(k, big);          // never removed
button.addListener(this);   // never detached

// Watching it:
java -Xlog:gc -Xmx512m App  // was -verbose:gc
```

USE IT WHEN Sizing the heap (`-Xmx`), reading GC logs when response times spike, and finding why memory keeps growing (a heap histogram with `jcmd <pid> GC.class_histogram`, or a heap dump opened in Eclipse MAT or VisualVM).

DON'T Don't call `System.gc()` from application code (it forces an expensive full collection and the JVM knows better when to run), and don't rely on `finalize()` to release resources (deprecated for removal since Java 18, JEP 421): use try-with-resources or `java.lang.ref.Cleaner`.

DO Remove references from long-lived structures (caches, static registries, listener lists) when you no longer need the object, and set the heap size explicitly in containers (`-Xmx` or `-XX:MaxRAMPercentage`).

REMEMBER *GC frees what is unreachable, not what is unused. A leak is a reference you forgot.*

IN DEPTH

What "reachable" means. The collector starts from the *roots*: local variables of every running thread, static fields of loaded classes, and references held by native code. Anything a root points to is alive; anything those objects point to is alive; and so on. Everything else is garbage. Note what this does *not* say: an object you will never use again but still have a reference to is not garbage. That is a memory leak in Java: a `static Map` used as a cache that only ever grows, a listener registered on a long-lived component and never removed, a `ThreadLocal` never cleared in a thread pool. The fix is always the same: drop the reference.

How collectors work, in outline. Marking: walk from the roots and mark every reachable object. Sweeping or copying: reclaim the rest, often by copying the survivors to a fresh area so the free memory is one contiguous block (which also makes allocation fast: just bump a pointer). Most collectors are *generational*, based on the observation that most objects die young: new objects go into a small young generation that is collected often and quickly; the few that survive several collections are promoted to the old generation, which is collected rarely.

Pauses, and the collectors you will hear about. Early collectors stopped every application thread for the whole collection ("stop the world"), which on a large heap meant pauses of seconds. The history since is a series of trade-offs between pause time and throughput: Serial (one thread, small heaps), Parallel (many threads, still full pauses, best raw throughput), CMS (concurrent marking, removed in Java 14), G1 (the default since Java 9: collects the heap in regions and aims for a pause target), ZGC and Shenandoah (production since Java 15: pauses of a few milliseconds regardless of heap size, at some throughput cost; ZGC became generational in Java 21). You rarely need to choose; you do need to know which one is running when you read a GC log.

What you control. The maximum heap (`-Xmx2g` , or `-XX:MaxRAMPercentage=75` inside a container), the collector (`-XX:+UseZGC`), and the logging (`-Xlog:gc` for one line per collection, `-Xlog:gc*` for detail). The signs of trouble: pauses growing over time, collections becoming more frequent while reclaiming less, and finally `OutOfMemoryError: Java heap space`. A heap histogram sorted by size usually names the leaking class in the first three lines.

WHY IT EXISTS

Manual memory management was the single largest source of crashes and security bugs in C and C++ programs, and it also made libraries hard to combine, because every API had to document who frees what. Garbage collection removes a whole class of bugs and the ownership bookkeeping with it. Lisp and Smalltalk had it decades earlier; Java brought it to mainstream business software.

TRY IT YOURSELF

Save the program below. Run it with a tiny heap and GC logging to watch collections happen; then uncomment the `keep.add` line to create a real leak and watch it die with `OutOfMemoryError`. While it runs, `jcmd <pid> GC.class_histogram | head` in another terminal shows what fills the heap (`jcmd` alone lists the pids).

```java
import java.util.*;
public class Churn {
    static List<byte[]> keep = new ArrayList<>();
    public static void main(String[] a) throws Exception {
        for (int i = 0; ; i++) {
            byte[] b = new byte[1_000_000];   // garbage next iteration
            // keep.add(b);                   // uncomment: a leak
            if (i % 1000 == 0) Thread.sleep(50);
        }
    }
}
// $ java -Xmx64m -Xlog:gc Churn.java
```

REFERENCES

JVMS section 2.5.3: the heap · `docs.oracle.com/javase/specs/jvms/se21/html/jvms-2.html#jvms-2.5.3`

HotSpot Garbage Collection Tuning Guide (Java SE 21) · `docs.oracle.com/en/java/javase/21/gctuning/introduction-garbage-collection-tuning.html`

JEP 248: Make G1 the default garbage collector (Java 9) · `openjdk.org/jeps/248`

JEP 363: Remove the CMS garbage collector (Java 14) · `openjdk.org/jeps/363`

JEP 421: Deprecate finalization for removal (Java 18) · `openjdk.org/jeps/421`

jcmd command reference · `docs.oracle.com/en/java/javase/21/docs/specs/man/jcmd.html`

6 HotSpot JIT

Java 1.3 · HotSpot VM by default

Interpret first, compile what is hot

There are two ways to execute bytecode. An *interpreter* reads one instruction at a time and performs it: simple, starts instantly, but ten to fifty times slower than native code. A *compiler* translates the bytecode into native machine code once, after which the CPU runs it directly. HotSpot, the JVM that became the default in Java 1.3 and is still the one you run today, does both: it starts by interpreting, counts how often each method and loop runs, and when something becomes "hot" it compiles just that method to native code in the background, *just in time* (JIT). Because it compiles while the program runs, it can use facts an ahead-of-time compiler never has: which branches are actually taken, which concrete types actually show up, which methods are small enough to inline.

```
# see what gets compiled, and when
java -XX:+PrintCompilation App
# tiered (7+): C1 quickly, then C2 optimised
java -XX:TieredStopAtLevel=1 App  # fast start
# never time cold code; use JMH
```

USE IT WHEN Explaining why a loop speeds up after the first second, why micro-benchmarks lie, why the first requests to a freshly started service are slow, and why profiling only the first seconds of a run is misleading.

DO Benchmark with JMH (the OpenJDK micro-benchmark harness) and let the JVM warm up before you measure. Keep hot methods small so the JIT can inline them (the default limit is about 35 bytes of bytecode for an ordinary method, more for frequently called ones).

DON'T Don't hand-optimise Java the way you would C: manual inlining, caching `array.length` in a local, avoiding getters "for speed". HotSpot does all of that itself, and your version only hides the intent from the next reader.

REMEMBER *The JIT optimises the program it observes. Measure after warm-up, never before.*

IN DEPTH

Tiered compilation. Since Java 7 HotSpot has two compilers. C1 compiles quickly with light optimisation and leaves profiling counters in the code; C2 compiles slowly but aggressively, using the profile C1 gathered. A hot method typically goes interpreter → C1 → C2 over its first few thousand calls. The compiled code lives in the *code cache*, a separate memory area; `-XX:+PrintCompilation` prints one line each time a method is compiled, with its tier.

What the optimiser does. *Inlining*: copies a small called method's body into the caller, removing the call and enabling every other optimisation across the boundary (this is why getters cost nothing). *Devirtualisation*: if only one implementation of an interface method has ever been seen, call it directly instead of looking it up. *Escape analysis*: if an object never leaves the method, allocate it on the stack or in registers rather than the heap. *Loop unrolling*, *dead code elimination*, *bounds-check elimination*: the classic compiler toolbox, applied with run-time knowledge.

Deoptimisation. Optimistic assumptions can break: a second implementation of that interface gets loaded, a branch that was never taken suddenly is. HotSpot then throws the compiled code away, falls back to the interpreter for that method, and recompiles later with the new facts. You see this as a short slowdown; it is the price of optimising on evidence.

Consequences for you. A JVM program is slower in its first seconds than in steady state (warm-up), so performance tests must run long enough, and health checks should not judge a service by its first request. Micro-benchmarks written as a loop with `System.nanoTime()` are usually wrong: the JIT may remove a computation whose result is unused, or hoist it out of the loop. JMH exists to prevent exactly these mistakes. Start-up and warm-up cost also motivated class data sharing (Java 10) and the ahead-of-time cache in Java 24 and 25 (chapter 12), which record what a training run learned and replay it on the next start.

WHY IT EXISTS

Java 1.0 interpreted everything and earned a reputation for being slow. Sun bought the HotSpot VM (from a team that had built the Strongtalk Smalltalk VM) and made it the default in 1.3. Its bet, that a compiler with run-time information can beat a static compiler on long-running programs, is why Java became acceptable for high-performance servers and why "Java is slow" stopped being true around 2000.

TRY IT YOURSELF

Run the program below with `-XX:+PrintCompilation`. The lines that appear while it runs are methods being compiled (the number before the method name is the tier). The timings printed by the program itself show the same work getting faster: the first round runs interpreted, later rounds run C2 code. Then try `-Xint` (interpreter only) and compare.

```
public class Warm {
    static long work(int n) { long s = 0; for (int i = 0; i < n; i++) s += i % 7; return s; }
    public static void main(String[] a) {
        for (int round = 1; round <= 5; round++) {
            long t = System.nanoTime();
            work(50_000_000);
            System.out.println("round " + round + ": " + (System.nanoTime() - t) / 1_000_000 + " ms");
        }
    }
}
// $ java -XX:+PrintCompilation Warm.java | grep -E "round|Warm::work"
// $ java -Xint Warm.java        # interpreter only: every round slow
```

REFERENCES

HotSpot group at OpenJDK · `openjdk.org/groups/hotspot/`

HotSpot glossary of terms (JIT, tiered compilation, deoptimization) · `openjdk.org/groups/hotspot/docs/HotSpotGlossary.html`

java command reference: -XX options · `docs.oracle.com/en/java/javase/21/docs/specs/man/java.html`

JMH, the Java Microbenchmark Harness · `github.com/openjdk/jmh`

JEP 483: Ahead-of-time class loading and linking (Java 24) · `openjdk.org/jeps/483`

7 Threads and synchronized

Java 1.0 · java.lang.Thread · JLS ch. 17

Monitors built into every object

A thread is an independent flow of execution inside the same program, sharing the same heap. Java had them in the language from day one: create a `Thread` with a `Runnable`, call `start()`, and its `run()` method executes concurrently with yours. Sharing the heap is the convenience and the danger: when two threads read and write the same variable without coordination, the result depends on timing. The `synchronized` keyword is Java's built-in answer: every object has a hidden lock (a *monitor*), and only one thread at a time can be inside `synchronized` blocks that lock the same object.

```
class Counter {
    private int n;
    synchronized void inc() { n++; }  // atomic
    synchronized int get() { return n; }
}
Thread t = new Thread(() -> counter.inc());
t.start();          // NOT t.run()
t.join();
```

USE IT WHEN Doing work in the background and using several CPU cores. Today you reach for an `ExecutorService` (Java 5, chapter 2) or virtual threads (Java 21, chapter 10); the raw `Thread` API is for understanding what those are built on.

DON'T Don't call `run()` when you mean `start()` (it runs the body on your own thread, no concurrency at all), don't synchronize on `String` literals or boxed numbers (they are shared across the whole JVM, so unrelated code may lock the same object), and don't touch `Thread.stop`: it has thrown `UnsupportedOperationException` since Java 20.

DO Guard every access to shared mutable state, reads included, with the same lock, or make the state immutable so there is nothing to guard. Prefer a private lock object (`private final Object lock = new Object()`) over `synchronized (this)`, so outside code cannot interfere with your locking.

REMEMBER *synchronized = mutual exclusion + visibility. You get both, or you have neither.*

IN DEPTH

Why `n++` is not safe. It looks like one operation but is three: read `n`, add one, write `n` back. If two threads both read 5, both add one, and both write 6, one increment is lost. Run two threads doing a million increments each without `synchronized` and you will get a total well short of two million, and a different number every run. This is a *race condition*; the exercise `Ex03_Regex_NIO_Threads` makes you see it.

What `synchronized` gives you. Two things. *Mutual exclusion*: while one thread holds the monitor of an object, every other thread that tries to enter a `synchronized` block on the same object waits. *Visibility*: when a thread releases the lock, all its writes become visible to the next thread that acquires it (the Java Memory Model calls this a happens-before relationship). Without the second guarantee a thread could keep reading a stale copy of a variable from its CPU cache forever; `volatile` gives visibility alone, without exclusion.

wait and notify. Also on every object: a thread holding the monitor can `wait()`, which releases the lock and sleeps until another thread holding the same monitor calls `notify()` or `notifyAll()`. This is the primitive under every blocking queue and condition. It is easy to get wrong (always call `wait` in a loop re-checking the condition); since Java 5 you use `BlockingQueue`, `CountDownLatch` or `Condition` instead.

The 1.0 rough edges. The original memory model was specified so loosely that correct-looking idioms such as double-checked locking were broken on real hardware; JSR 133 rewrote it for Java 5 (chapter 2). `Thread.stop`, `suspend` and `resume` could leave locks held or objects half-updated; they were deprecated in 1.2 and now throw. The design that survived, a monitor in every object, is also what virtual threads (Java 21) still use.

WHY IT EXISTS

Java was designed for programs that wait on networks and respond to users: a GUI must stay responsive while a file loads, a server must talk to many clients at once. Building threads into the language, rather than leaving them to an operating-system library as C did, meant every Java program on every platform used the same model.

TRY IT YOURSELF

Run the program below twice: once as is (unsynchronised), once with `synchronized` added to `inc()`. The first prints a different number under 2,000,000 each run; the second prints exactly 2,000,000. Then replace `t1.start()` with `t1.run()` and notice the program becomes sequential and slow but "correct": that is the `start()` versus `run()` trap.

```
public class Race {
    static int n;
    static void inc() { n++; }          // add synchronized here
    public static void main(String[] a) throws Exception {
        Runnable r = () -> { for (int i = 0; i < 1_000_000; i++) inc(); };
        Thread t1 = new Thread(r), t2 = new Thread(r);
        t1.start(); t2.start();
        t1.join(); t2.join();
        System.out.println(n);         // < 2000000 without the lock
    }
}
```

REFERENCES

Java Language Specification, chapter 17, Threads and Locks · `docs.oracle.com/javase/specs/jls/se21/html/jls-17.html`

Java Tutorials: Concurrency · `docs.oracle.com/javase/tutorial/essential/concurrency/index.html`

JSR 133: Java Memory Model and Thread Specification revision (Java 5) · `jcp.org/en/jsr/detail?id=133`

Thread API (stop, suspend, resume removed behaviour) · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html`

JEP 444: Virtual threads (Java 21) · `openjdk.org/jeps/444`

3 Java 1.1: looking inside

Classes inside classes, types at run time, objects on the wire.

8 Inner and anonymous classes

Java 1.1 · JLS §8.1.3, §15.9.5

A class inside a class, capturing it

Java 1.0 only allowed classes at the top level of a file. Java 1.1 allowed a class to be declared inside another class, inside a method, or even inside an expression, and gave the four resulting kinds different powers. A *static nested* class is simply a class that lives in another class's namespace (`Map.Entry`). An *inner* class (non-static) is tied to an instance of the outer class and can use that instance's fields and methods directly. A *local* class is declared inside a method body. An *anonymous* class has no name at all: it is declared and instantiated in a single expression, `new Runnable() { ... }` , to make a one-off implementation on the spot.

```
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        save();        // sees outer this
    }
});
static class Node { ... }      // no outer ref
class Iter implements Iterator<T> {} // inner
```

USE IT WHEN A helper type that only makes sense inside one class (a linked list's `Node` , a collection's `Iterator`). Anonymous classes today only where a lambda cannot go: an interface with more than one method, or an object that needs fields of its own.

DON'T Don't hand an inner-class or anonymous-class instance to something long-lived (a cache, a static registry, a listener you never remove): through its hidden `Outer.this` it keeps the whole outer object, and everything that object references, from being garbage collected. This is one of the classic Java memory leaks.

DO Declare nested classes `static` unless they genuinely need the enclosing instance. An inner class keeps a hidden reference to its outer object, which keeps that object alive and drags it into serialization.

REMEMBER *Inner = carries Outer.this. Static nested = just lives there. When in doubt, static.*

IN DEPTH

The four kinds, with the rule for each. *Static nested*: `class Outer { static class Nested {} }` , created with `new Outer.Nested()` ; it has no access to `Outer` 's instance fields, it is just grouping and visibility. *Inner*: `class Outer { class Inner {} }` ; every `Inner` object belongs to one `Outer` object and is created from it, `outer.new Inner()` (or just `new Inner()` from inside an `Outer` method). Inside `Inner` , `Outer.this` names that enclosing object, and outer fields can be used by name. *Local*: declared inside a method, visible only there; it may use the method's parameters and local variables if they are effectively final. *Anonymous*: `new SomeInterface() { ...body... }` or `new SomeClass(args) { ...overrides... }` ; it is a local class without a name, so it can only be instantiated at that one spot.

The capture rule, and why it exists. A local or anonymous class may read local variables of the enclosing method, but only if they are never reassigned (before Java 8 you had to write `final` explicitly; since Java 8 "effectively final" is enough). The reason is mechanical: the compiler *copies* the variable's value into a hidden field of the nested class when it is created. If the method could change the variable afterwards, the copy and the original would disagree, so Java forbids the change. Lambdas inherit exactly this rule.

What the compiler produces. Each nested class becomes its own class file: `Outer$Inner.class` , `Outer$1.class` for the first anonymous class, and so on. Look at `javap -p Outer$Inner` and you will see a field named `this$0` : that is the hidden reference to the enclosing object, and it is exactly what causes the memory leak described above. A static nested class has no such field.

Anonymous classes before lambdas. For seventeen years this was how Java passed behaviour around. Every Swing button handler, every `Comparator` given to `Collections.sort` , every `Runnable` handed to a thread was an anonymous class, five lines of ceremony around one line of logic. Java 8 lambdas (chapter 4) replace anonymous classes for any interface with a single abstract method, with one subtle difference: inside a lambda, `this` means the enclosing object; inside an anonymous class, `this` means the anonymous object itself. Anonymous classes remain the right tool when the interface has several methods or the object needs its own state.

WHY IT EXISTS

Java 1.1 introduced a new GUI event model in which every button, menu and text field notified *listener* objects. Without nested classes, every handler would have been a separate top-level class in its own file, far from the code it belonged to. Inner and anonymous classes let the handler sit next to the component it serves and reach the enclosing window's fields directly.

TRY IT YOURSELF

In `jshell` , build an outer class with an inner and a static nested class and watch what the compiler allows. Then save the same code to a file, compile it, and list the class files to see `Outer$Inner.class` ; `javap -p 'Outer$Inner'` shows the hidden `this$0` field.

```
jshell> class Outer { int x = 42;
   ...>   class Inner { int peek() { return x; } }            // sees x
   ...>   static class Nested { /* int peek() { return x; } */ } // would not compile
   ...> }
jshell> new Outer.Nested()            // fine: no outer instance needed
jshell> new Outer.Inner()             // error: needs an enclosing instance
jshell> Outer o = new Outer(); o.new Inner().peek()   // 42

$ javac Outer.java && ls Outer*.class && javap -p 'Outer$Inner'
```

REFERENCES

JLS section 8.1.3: inner classes and enclosing instances · `docs.oracle.com/javase/specs/jls/se21/html/jls-8.html#jls-8.1.3`

JLS section 15.9.5: anonymous class declarations · `docs.oracle.com/javase/specs/jls/se21/html/jls-15.html#jls-15.9.5`

Java Tutorials: Nested classes · `docs.oracle.com/javase/tutorial/java/javaOO/nested.html`

Java Tutorials: Anonymous classes · `docs.oracle.com/javase/tutorial/java/javaOO/anonymousclasses.html`

JEP 126: Lambda expressions (Java 8), the successor for single-method cases · `openjdk.org/jeps/126`

9 Reflection and JavaBeans

Java 1.1 · java.lang.reflect, java.beans · proxies 1.3

Inspect and call types at run time

Normally the compiler must know every class and method you use. Reflection lets a running program discover them instead: given any object, ask for its `Class`, list its fields and methods, read a field, call a method by its *name as a string*, or create an instance of a class whose name arrived in a configuration file. The `java.lang.reflect` package (`Class`, `Method`, `Field`, `Constructor`) is the machinery. JavaBeans is a naming convention layered on top: a public no-argument constructor and `getX()` / `setX()` pairs, so that a tool can find a class's "properties" by reflection without being told.

```
Class<?> c = Class.forName("com.acme.Invoice");
Object o = c.getConstructor().newInstance();
Method m = c.getMethod("setId", long.class);
m.invoke(o, 42L);
for (Field f : c.getDeclaredFields())
    System.out.println(f.getName());
// 1.3: Proxy.newProxyInstance(cl, ifaces, h)
```

USE IT WHEN Writing frameworks, serialisers, dependency injectors, test tools and plug-in loaders: code that must work with classes it has never seen. In ordinary application code almost never: a `switch` or an interface is clearer, type-checked and faster.

DO Cache `Method` and `Field` objects once you have looked them up (the lookup is slow, the call is not), and prefer `MethodHandle` (Java 7) or code generation at build time when performance matters.

DON'T Don't `setAccessible(true)` your way into JDK internals (`sun.misc.*`, private fields of `java.lang` classes). Since Java 16 the module system blocks it at run time (JEP 396, JEP 403) unless the user adds `--add-opens` flags, which is exactly what you want to avoid asking them to do.

REMEMBER *Reflection trades compile-time checks for run-time flexibility. Frameworks pay that price so you don't.*

IN DEPTH

Where reflection shows up in your day. Spring creates your `@Service` objects and injects their dependencies by reading constructors and fields reflectively. Jackson maps JSON to objects by finding getters and setters (the JavaBeans convention) or record components. JPA/Hibernate reads `@Column` annotations and sets private fields directly. JUnit finds methods annotated `@Test` and invokes them. The JDBC `Class.forName("org.postgresql.Driver")` incantation of old was reflection loading a class by name. None of these libraries knows your classes at compile time; reflection is how they work anyway.

JavaBeans in one paragraph. A JavaBean is a class with a public no-argument constructor, private fields exposed through `getName()` / `setName(String)` pairs (`isActive()` for booleans), optionally `Serializable`, and optionally firing `PropertyChangeEvent`s when a property changes. The convention was invented so that visual GUI builders could show a component's properties in a panel; it outlived the GUI builders because every mapping framework adopted the same getter/setter rule. Records (Java 16) are the modern alternative for pure data: immutable, no setters, accessor named `name()`.

Dynamic proxies (1.3). `Proxy.newProxyInstance` manufactures, at run time, a class that implements any interfaces you name and forwards every method call to one `InvocationHandler`. With twenty lines you get an object that logs every call, retries on failure, or opens a transaction around a method, without writing a class per interface. Spring's `@Transactional`, remote stubs, and much of Mockito rest on this idea.

The costs. Reflective calls are slower than direct ones (less so since Java 18 reimplemented them on method handles, JEP 416), they bypass compile-time type checking (a typo in `"setId"` fails at run time), they break encapsulation, and they make code hard to follow in an IDE. The module system (Java 9) added a second limit: a class in another module can only be reflected into if that module `opens` the package. For the JDK's own packages this became the default in Java 16, closing the door on a decade of libraries poking at `sun.misc.Unsafe` and friends.

WHY IT EXISTS

Component software was the goal of the mid-1990s: assemble applications from reusable parts in a visual builder, the way Visual Basic did. For a builder to work with a component it had never seen, it needed to ask the component about itself at run time. Reflection answered that, JavaBeans standardised the questions, and frameworks found the combination indispensable for entirely different reasons.

TRY IT YOURSELF

In `jshell`, inspect a JDK class you know, call a method by name, and build a proxy that announces every call. Then introduce a typo in the method name string and see how late the failure arrives.

```
jshell> var c = Class.forName("java.util.ArrayList")
jshell> c.getMethods().length                       // dozens of public methods
jshell> var list = c.getConstructor().newInstance()
jshell> c.getMethod("add", Object.class).invoke(list, "hi"); list   // [hi]
jshell> c.getMethod("addd", Object.class)           // NoSuchMethodException, at run time

jshell> import java.lang.reflect.*;
jshell> Runnable real = () -> System.out.println("working");
jshell> Runnable loud = (Runnable) Proxy.newProxyInstance(Runnable.class.getClassLoader(),
   ...>     new Class<?>[]{Runnable.class},
   ...>     (p, m, args) -> { System.out.println("calling " + m.getName()); return m.invoke(real, args); });
jshell> loud.run()                                   // calling run / working
```

REFERENCES

Java Tutorials: The Reflection API · `docs.oracle.com/javase/tutorial/reflect/index.html`

java.lang.reflect package · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/reflect/package-summary.html`

java.lang.reflect.Proxy API (dynamic proxies) · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/reflect/Proxy.html`

JavaBeans specification 1.01 · `www.oracle.com/java/technologies/javase/javabeans-spec.html`

JEP 403: Strongly encapsulate JDK internals (Java 17) · `openjdk.org/jeps/403`

JEP 416: Reimplement core reflection with method handles (Java 18) · `openjdk.org/jeps/416`

10 Serialization

Java 1.1 · java.io.Serializable

Objects to bytes, with a bill

Serialization turns a live object, and every object it references, into a stream of bytes that can be written to a file or sent over a network, and later turned back into objects. In Java 1.1 this is almost free to use: mark the class `implements Serializable` (an interface with no methods), hand the object to `ObjectOutputStream.writeObject`, and read it back with `ObjectInputStream.readObject`. The mechanism copies private fields directly and rebuilds objects *without running their constructors*, which is both why it works for any class and why it is dangerous.

```
class Session implements Serializable {
    static final long serialVersionUID = 1L;
    transient Connection db;      // not written
    String user;
}
// read with a filter (JEP 290, Java 9)
ObjectInputFilter f = ObjectInputFilter.Config
    .createFilter("com.acme.*;!*");
```

USE IT WHEN Only when a protocol forces it on you: RMI, old servlet containers replicating HTTP sessions, JMX, some caching libraries. For storing or sending your own data use JSON, Protocol Buffers or any format with an explicit schema and versioning.

DO If you must use it: declare `serialVersionUID` explicitly (otherwise the compiler derives one from the class shape and any edit breaks old streams), mark passwords and connections `transient` so they are not written, and install a deserialization filter (`-Djdk.serialFilter=com.acme.*;!*`) that allows only your own classes.

DON'T Never deserialize bytes that arrived from a network or a user without a filter, and don't add `implements Serializable` "in case it is needed": it turns your private fields into a public file format you must support forever.

REMEMBER *Serializable makes your private fields a wire format and your classpath an attack surface.*

IN DEPTH

The format. The stream contains the class name, the `serialVersionUID`, and each non-transient, non-static field's name and value, recursively for referenced objects (shared references are written once and back-referenced, so cycles work). A class can customise this with private `writeObject` / `readObject` methods or replace itself with `writeReplace` / `readResolve`. All of this is defined in the Java Object Serialization Specification; it has not changed in essentials since 1.1.

The security problem, in plain terms. `ObjectInputStream` will instantiate *any* `Serializable` class that is on the classpath and named in the incoming bytes, and will run that class's `readObject` method. An attacker who can send you a stream therefore chooses which classes get created and in what arrangement. In 2015 researchers showed that ordinary library classes (Apache Commons Collections among them) could be chained so that deserializing a crafted stream executed an arbitrary command: a *gadget chain*. Products that accepted serialized objects over the network (WebLogic, JBoss, Jenkins, and many in-house systems) were remotely exploitable without any bug in their own code. The lesson generalises: deserializing untrusted input is executing untrusted input.

The brittleness problem. The byte format is derived from the fields, so renaming a field, changing its type, or moving a class breaks every stored stream unless you hand-write compatibility code. Teams who used serialization for long-term storage discovered they could no longer refactor their domain model.

What the JDK did about it. Java 9 added deserialization filters (JEP 290): an allow/deny list of classes plus limits on graph depth and array sizes, set per stream or JVM-wide. Java 17 made filters configurable per context (JEP 415). Records (Java 16) serialize through their canonical constructor, so their invariants hold even for data from a stream. The platform's architects have said they would like to replace the mechanism altogether; until then, treat it as legacy.

WHY IT EXISTS

In 1997 Java needed a built-in way to move objects between JVMs for Remote Method Invocation and to save them to disk, years before JSON, XML binding or Protocol Buffers existed. Making it automatic for every class was the most convenient design possible, and convenience is precisely what made it unsafe.

TRY IT YOURSELF

Serialize a small object, look at the bytes, then deserialize it with a filter that refuses the class. The hex dump shows the class name in clear text, which is how an attacker knows what to target; the filter is the one-line defence.

```
import java.io.*;
public class Ser {
    record Ticket(String id, int seats) implements Serializable {}
    public static void main(String[] a) throws Exception {
        var bytes = new ByteArrayOutputStream();
        try (var out = new ObjectOutputStream(bytes)) { out.writeObject(new Ticket("A1", 2)); }
        System.out.println(new String(bytes.toByteArray(), "ISO-8859-1").replaceAll("[^\\x20-\\x7e]", "."));
        var in = new ObjectInputStream(new ByteArrayInputStream(bytes.toByteArray()));
        in.setObjectInputFilter(ObjectInputFilter.Config.createFilter("java.base/*;!*"));
        System.out.println(in.readObject());        // InvalidClassException: filter status REJECTED
    }
}
```

REFERENCES

Java Object Serialization Specification (Java SE 21) · `docs.oracle.com/en/java/javase/21/docs/specs/serialization/index.html`

Java Serialization Filtering guide · `docs.oracle.com/en/java/javase/21/core/serialization-filtering.html`

JEP 290: Filter incoming serialization data (Java 9) · `openjdk.org/jeps/290`

JEP 415: Context-specific deserialization filters (Java 17) · `openjdk.org/jeps/415`

JEP 395: Records (Java 16), section on serialization · `openjdk.org/jeps/395`

4 Java 1.2: collections

Interfaces for groups of things, implementations you can swap.

11 Collections framework

Java 1.2 · java.util

List, Set, Map: interfaces first

Before 1.2, Java had arrays (fixed size), `Vector` (a growable array) and `Hashtable` (a key-value table), and no common vocabulary between them. The collections framework replaced this with a small set of *interfaces* describing kinds of collection (`Collection`, `List`, `Set`, `Map`, and the sorted variants) and a set of interchangeable *implementations* for each (`ArrayList` and `LinkedList` for `List`; `HashSet`, `LinkedHashSet` and `TreeSet` for `Set`; `HashMap`, `LinkedHashMap` and `TreeMap` for `Map`). Code written against the interface works with any implementation, so choosing one becomes a local performance decision instead of something that shapes your whole API.

```
List<Order> os = new ArrayList<>(); // interface
Set<String> seen = new HashSet<>();   // no dups
Map<String, Order> byId = new HashMap<>();
SortedMap<String, Order> s = new TreeMap<>();
Set<String> ins = new LinkedHashSet<>(); // 1.4
// HashMap: O(1) avg. TreeMap: O(log n), sorted.
// Only the right-hand side ever changes.
```

USE IT WHEN Any group of things. `ArrayList` and `HashMap` by default; `TreeMap` or `TreeSet` when you need sorted order; `LinkedHashMap` for insertion order or a simple LRU cache.

DO Declare variables, fields, parameters and return types as the interface (`List`, not `ArrayList`), and make `equals` and `hashCode` correct and consistent on anything that goes into a `HashSet` or is used as a `HashMap` key.

DON'T Don't add to or remove from a collection while iterating over it with for-each (you get `ConcurrentModificationException`; use an `Iterator` or `removeIf`), don't use a mutable object as a map key or set element, and don't reach for `LinkedList` expecting it to be faster: `ArrayList` wins almost every real benchmark.

REMEMBER *Interface on the left, implementation on the right, and only the right side ever changes.*

IN DEPTH

The interfaces and what each promises. `Collection`: a group of elements you can add to, remove from, and iterate. `List`: ordered, allows duplicates, every element has an index. `Set`: no duplicates (as defined by `equals`), no promise about order. `SortedSet`: a set kept in sorted order. `Map`: not a `Collection` but a lookup table from unique keys to values; `SortedMap` keeps the keys sorted. Java 5 added `Queue`, Java 6 `Deque` and `NavigableMap`, Java 21 `SequencedCollection` for anything with a defined first and last.

How to choose an implementation. `ArrayList` is a growable array: constant-time index access, cheap append, expensive insert in the middle. `HashMap` and `HashSet` are hash tables: average constant-time lookup, no order, and they *require* correct `hashCode` and `equals` on the key type, since the hash decides which bucket to look in. `LinkedHashMap` and `LinkedHashSet` (1.4) are hash tables that also remember insertion order. `TreeMap` and `TreeSet` are red-black trees: everything is logarithmic time and always sorted, by `Comparable` or a `Comparator` you supply. `LinkedList` is almost never the right answer; `ArrayDeque` is the better queue and stack.

The mutable-key trap. A `HashMap` stores an entry in the bucket chosen by the key's `hashCode` at insertion time. If you later change a field of that key object, its hash changes, and the entry is now in the wrong bucket: `containsKey` returns `false` for a key that is demonstrably in the map. Keys should be immutable; strings, numbers and records are safe.

What came later. Generics (Java 5, chapter 2) made collections type-safe, so `List<Order>` replaced `List` with casts everywhere. `java.util.concurrent` (Java 5) added collections safe to share between threads. Streams (Java 8) added a query language over any collection. `List.of`, `Set.of` and `Map.of` (Java 9) create immutable collections in one call. The interfaces from 1998 absorbed all of it without a rewrite, which is the strongest evidence that the design was right.

WHY IT EXISTS

Joshua Bloch designed the framework at Sun after seeing every project reinvent lists and hash tables with incompatible APIs. One shared vocabulary means a library can accept any `List` and a team can read any other team's code. The design is documented in a short FAQ that is still worth reading for the reasoning behind each decision.

TRY IT YOURSELF

In `jshell`, put the same elements into a `HashSet`, a `LinkedHashSet` and a `TreeSet` and print them to see the three orderings. Then demonstrate the mutable-key trap with a tiny mutable class.

```
jshell> var in = java.util.List.of("pear", "apple", "fig", "apple")
jshell> new java.util.HashSet<>(in)        // some order, 3 elements
jshell> new java.util.LinkedHashSet<>(in)  // [pear, apple, fig]
jshell> new java.util.TreeSet<>(in)        // [apple, fig, pear]

jshell> class Key { int v; Key(int v) { this.v = v; }
   ...>   public int hashCode() { return v; }
   ...>   public boolean equals(Object o) { return o instanceof Key k && k.v == v; } }
jshell> var k = new Key(1); var set = new java.util.HashSet<Key>(); set.add(k);
jshell> k.v = 2; set.contains(k)           // false: it is in the wrong bucket now
```

REFERENCES

Collections Framework overview (Java SE 21) · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/doc-files/coll-index.html`

Collections Framework design FAQ · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/doc-files/coll-designfaq.html`

Java Tutorials: Collections · `docs.oracle.com/javase/tutorial/collections/index.html`

12 Iterator, Comparator, utilities

Java 1.2 · java.util

Walk it, order it, wrap it

Three supporting pieces make the framework usable. `Iterator` is the one way to walk any collection: `hasNext()`, `next()`, and `remove()`, which deletes the element just returned and is the only safe way to delete while walking (the for-each loop added in Java 5 uses an `Iterator` underneath). `Comparable` lets a type define its own *natural* order in `compareTo` (strings alphabetically, numbers numerically); `Comparator` lets you supply any other order from outside, without touching the type. The `Collections` class holds the algorithms and wrappers that work on any collection: `sort`, `binarySearch`, `reverse`, `shuffle`, `max`, `unmodifiableList`, `synchronizedMap`, `emptyList`.

```
Iterator<Order> it = xs.iterator();
while (it.hasNext())
    if (it.next().cancelled) it.remove();
Comparator<Order> byQty = new Comparator<>() {
    public int compare(Order a, Order b) {
        return Integer.compare(a.qty, b.qty); }
};
Collections.sort(xs, byQty);    // stable sort
```

USE IT WHEN Deleting during iteration (`Iterator.remove`, or `removeIf` since Java 8), sorting by a key that is not the type's natural one, and handing out read-only views of internal lists from a getter.

DO Implement `Comparable` only for the one obvious order of a type, and keep it consistent with `equals` (if `compareTo` returns 0, `equals` should be true); everything else is a `Comparator`. Return `Collections.unmodifiableList(inner)` from getters, never the field itself.

DON'T Don't compute `a.x - b.x` in a comparator (it overflows for large values and gives the wrong sign; use `Integer.compare`), and don't believe `Collections.synchronizedMap` makes a check-then-put sequence atomic: it locks each call, not the pair.

REMEMBER *Comparable = the type's own order. Comparator = an order you bring. Collections = the toolbox.*

IN DEPTH

Fail-fast iteration. The JDK's iterators detect that the collection was structurally modified behind their back (an element added or removed by anything other than the iterator itself) and throw `ConcurrentModificationException` on the next step. Despite the name, this usually has nothing to do with threads; the classic cause is calling `list.remove(x)` inside a for-each over `list`. The fix is `Iterator.remove()`, or `list.removeIf(predicate)` since Java 8, or collecting what to delete and removing afterwards.

The comparison contract. `compareTo` and `compare` return a negative number, zero or a positive number for less-than, equal and greater-than. The contract requires consistency (if a < b and b < c then a < c) and symmetry; violate it and `TreeMap` loses entries and `sort` throws `IllegalArgumentException: Comparison method violates its general contract` (Java 7's TimSort checks). Subtracting two ints breaks the contract silently when the difference overflows, which is why `Integer.compare` exists. Java 8 added `Comparator.comparing(keyExtractor)`, `thenComparing` and `reversed` (chapter 4), which make most hand-written comparators unnecessary.

Views, not copies. `Collections.unmodifiableList(inner)` returns a *view*: calling `add` on it throws, but changes made to `inner` afterwards are visible through it. That is exactly what you want for a getter (callers cannot modify your list, but they always see the current state) and exactly what surprises people who expected a snapshot. `List.copyOf` (Java 10) makes a true immutable copy. `Arrays.asList` is also a view, over the array: fixed size, `add` throws, but `set` writes through to the array.

The algorithms. `Collections.sort` is stable (equal elements keep their relative order), which matters when you sort by one key after another. `binarySearch` requires a sorted list. `shuffle`, `reverse`, `rotate`, `frequency`, `disjoint` and `nCopies` are the small helpers people rewrite without knowing they exist; the `Arrays` class has the matching set for arrays.

WHY IT EXISTS

Algorithms written once against the `List` interface work for every list implementation, past and future, and a single `Iterator` protocol means a loop never has to know what it is walking over. Separating `Comparable` (the type's own idea of order) from `Comparator` (the caller's) lets the same objects be sorted ten ways without ten subclasses.

TRY IT YOURSELF

Trigger the fail-fast exception on purpose, fix it with an iterator, then see how an unmodifiable view differs from a copy.

```
jshell> var xs = new java.util.ArrayList<>(java.util.List.of(1, 2, 3, 4))
jshell> for (int x : xs) if (x % 2 == 0) xs.remove((Integer) x);   // ConcurrentModificationException
jshell> for (var it = xs.iterator(); it.hasNext(); ) if (it.next() % 2 == 0) it.remove();
jshell> xs                                                        // [1, 3]

jshell> var view = java.util.Collections.unmodifiableList(xs); var copy = java.util.List.copyOf(xs);
jshell> xs.add(99); view                                          // [1, 3, 99]: the view follows
jshell> copy                                                      // [1, 3]: the copy does not
jshell> view.add(5)                                               // UnsupportedOperationException
```

REFERENCES

Iterator API (fail-fast behaviour described on ArrayList) · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Iterator.html`

Comparable and Comparator API · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Comparator.html`

Collections utility class API · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Collections.html`

Java Tutorials: Algorithms (Collections) · `docs.oracle.com/javase/tutorial/collections/algorithms/index.html`

13 Vector, Hashtable, Enumeration ~~DEPRECATED~~

Java 1.0 · java.util

The 1.0 collections: legacy, avoid

`Vector` (a growable array), `Hashtable` (a key-value table), `Stack` (a `Vector` subclass) and `Enumeration` (the original way to walk them) are the collection classes of Java 1.0, written before the framework existed. When the framework arrived in 1.2 they were retrofitted to implement `List` and `Map` so old code kept working, but their design shows its age: every method is `synchronized`, which costs time on every call and still does not make multi-step operations safe; `Stack` exposes the whole `Vector` API; `Hashtable` refuses `null` keys and values. The JDK documentation calls them legacy classes. They are not formally deprecated, so you will not get a compiler warning, but every modern replacement is better.

```
Vector<String> v = new Vector<>();     // legacy
Hashtable<String, Integer> h = ...;    // legacy
Enumeration<String> e = v.elements();  // legacy
// instead:
List<String> l = new ArrayList<>();
Map<String, Integer> m = new HashMap<>();
Deque<String> stack = new ArrayDeque<>(); // 6
Map<K, V> safe = new ConcurrentHashMap<>(); // 5
```

USE IT WHEN Only to read old code or to satisfy an ancient API that demands them. `Properties` (a `Hashtable` subclass used for `.properties` configuration files) is the one you still meet regularly, and a few JDK methods still return an `Enumeration` (`NetworkInterface.getNetworkInterfaces`, `ZipFile.entries`).

DO Replace on sight: `Vector` → `ArrayList`, `Hashtable` → `HashMap`, or `ConcurrentHashMap` if threads share it, `Stack` → `ArrayDeque`, `Enumeration` → `Iterator` (`Collections.list(e)` or `e.asIterator()` since Java 9 convert).

DON'T Don't choose `Vector` or `Hashtable` "because they are thread-safe": a lock per method call does not make `if (!v.contains(x)) v.add(x)` atomic, since another thread can add between your two calls. `java.util.concurrent` (chapter 2) solves the actual problem with `putIfAbsent` and friends.

REMEMBER *Hashtable's lowercase t is the tell: it predates the naming convention and the framework.*

IN DEPTH

Why per-method locking is not thread safety. `synchronized` on each method guarantees that a single call completes without interference. Real code combines calls: check, then act. Two threads both run `contains(x)`, both see `false`, both call `add(x)`, and the "thread-safe" vector now holds `x` twice. Making that correct needs one lock held across both calls (`synchronized (v) { ... }`) or an operation designed to be atomic, such as `ConcurrentHashMap.putIfAbsent`. Meanwhile, single-threaded code using `Vector` pays for a lock it never needed on every call.

The Stack problem, again. `Stack extends Vector` was the Java 1.0 team's own example of inheriting for code reuse: a stack is implemented on top of a growable array, so why not extend it? The answer is that every `Vector` method, including `insertElementAt`, `removeElementAt` and `get(index)`, became a `Stack` method, and a stack whose middle anyone can edit is not a stack. The class cannot be fixed without breaking programs, so the JDK's own documentation for `Stack` says to use `Deque` (`ArrayDeque`) instead.

Enumeration versus Iterator. `Enumeration` has `hasMoreElements` and `nextElement`; `Iterator` has shorter names and adds `remove`. The newer one won, and `Enumeration` survives only in a handful of old JDK signatures. Since Java 9, `enumeration.asIterator()` bridges the two.

WHY IT EXISTS

These classes were written in 1995 for a language without interfaces for collections, before anyone had designed a framework, and with the belief that making everything synchronized would make concurrent programming safe. The collections framework (1.2) and `java.util.concurrent` (5) each corrected one of those two assumptions.

TRY IT YOURSELF

See the three legacy behaviours in `jshell`: the stack whose middle you can edit, the table that rejects null, and the enumeration bridge.

```
jshell> var st = new java.util.Stack<Integer>(); st.push(1); st.push(2);
jshell> st.insertElementAt(42, 0); st.pop()      // 2, but the "stack" is now [42, 1]
jshell> new java.util.Hashtable<String,String>().put("k", null)   // NullPointerException
jshell> new java.util.HashMap<String,String>().put("k", null)     // fine (returns null)
jshell> java.util.Collections.list(java.net.NetworkInterface.getNetworkInterfaces()).size()  // Enumeration -> Li
```

REFERENCES

Vector API (notes it is a legacy class) · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Vector.html`

Stack API (recommends Deque instead) · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Stack.html`

Hashtable API · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Hashtable.html`

Deque API (ArrayDeque as stack and queue) · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Deque.html`

5 Java 1.4: the toolbox

Text, I/O and diagnostics finally in the standard library.

14 Regular expressions

Java 1.4 · JSR 51 · java.util.regex

Compile once, match many

A regular expression is a small language for describing text patterns: `\d{4}-\d{2}-\d{2}` means "four digits, a dash, two digits, a dash, two digits". Java 1.4 added Perl-style regular expressions to the standard library in two classes. `Pattern.compile(regex)` parses the expression once into an efficient internal form; `pattern.matcher(text)` creates a `Matcher` that runs it over one input, with `matches()` (does the whole input fit?), `find()` (is there a next occurrence?), `group(n)` (what did the n-th parenthesised part capture?) and `replaceAll`. The convenience methods `String.matches`, `split` and `replaceAll` are shortcuts that compile the expression again on every call.

```
static final Pattern DATE = Pattern.compile(
    "(\\d{4})-(\\d{2})-(\\d{2})");

Matcher m = DATE.matcher(text);
while (m.find())
    System.out.println(m.group(1));  // the year
boolean ok = DATE.matcher(s).matches(); // whole
String[] parts = s.split("\\s*,\\s*");
```

USE IT WHEN Validating, extracting and splitting text that has a regular structure: dates, identifiers, log lines, simple CSV. Not for HTML, nested brackets, or anything with recursive structure, which needs a real parser.

DO Compile once into a `static final Pattern`, name your groups (`(?<year>\d{4})`, since Java 7) so `m.group("year")` reads better than `m.group(1)`, and anchor with `^` and `$` or use `matches()` when you mean the whole string.

DON'T Don't call `String.matches`, `split` or `replaceAll` inside a hot loop (each call recompiles the pattern), and don't run nested quantifiers such as `(a+)+` on input you do not control: on a non-matching string the engine tries exponentially many paths, and a few dozen characters can freeze a thread for minutes (a denial-of-service known as catastrophic backtracking).

REMEMBER *Pattern is the compiled program, Matcher is one run of it. matches() = all of it, find() = somewhere in it.*

IN DEPTH

The syntax you need most. Literal characters match themselves. `.` any character; `\d` digit, `\w` word character, `\s` whitespace (capitals negate: `\D` non-digit); `[a-z]` a character class, `[^0-9]` a negated one. Quantifiers: `*` zero or more, `+` one or more, `?` optional, `{3}` exactly three, `{2,5}` two to five; add `?` to make a quantifier lazy (shortest match). Anchors: `^` start, `$` end, `\b` word boundary. `(...)` groups and captures, `(?:...)` groups without capturing, `a|b` alternation. Flags such as `Pattern.CASE_INSENSITIVE` and `MULTILINE` are passed to `compile`.

Backslashes. In a Java string literal a backslash must itself be escaped, so the regex `\d+` is written `"\\d+"`. Every regex beginner loses an hour to this. Text blocks (Java 15) do not help, since `\` is still an escape there; `Pattern.quote(s)` helps when you want to match a literal string that may contain special characters.

matches, find, lookingAt. `matches()` requires the entire input to match the pattern. `find()` scans forward for the next place where the pattern matches, and can be called repeatedly to walk through all occurrences. `lookingAt()` matches at the start without requiring the end. Most "my regex does not match" bugs are a `matches()` call where `find()` was meant, or the reverse.

Catastrophic backtracking, explained. The engine tries alternatives and backtracks when one fails. With a pattern like `(a+)+b` and an input of thirty `a`s followed by `c`, there are roughly 2^{30} ways to split the `a`s among the groups, and the engine tries all of them before giving up. The symptoms are a CPU core pinned at 100% by a single request. Avoid nested quantifiers over overlapping sets, prefer possessive quantifiers (`a++`) or atomic groups when you need them, and test any regex that touches user input against a long non-matching string.

WHY IT EXISTS

Text processing was Perl's home ground, and until 1.4 Java programmers either wrote `indexOf` loops or pulled in a third-party library (Jakarta ORO, gnu.regexp). JSR 51 brought a compatible engine into the standard library so that parsing a log line or validating an input field needed nothing extra.

TRY IT YOURSELF

Explore the three matching modes and the split behaviour in `jshell`, then time a catastrophic pattern on a short input (the last line may take a very long time: press Ctrl-C to abandon it).

```
jshell> var p = java.util.regex.Pattern.compile("(\\d{4})-(\\d{2})")
jshell> p.matcher("2026-10").matches()              // true: the whole input
jshell> p.matcher("due 2026-10 or 2027-01").matches() // false
jshell> var m = p.matcher("due 2026-10 or 2027-01"); m.find(); m.group(1)   // "2026"
jshell> m.find(); m.group(1)                        // "2027"
jshell> "a,b,,c".split(",").length                  // 4: inner empty string kept
jshell> "a,b,,".split(",").length                   // 2: trailing empties dropped
jshell> "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaac".matches("(a+)+b")   // very slow: backtracking
```

REFERENCES

Pattern API: the complete regex syntax supported by Java · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/regex/Pattern.html`

Java Tutorials: Regular Expressions · `docs.oracle.com/javase/tutorial/essential/regex/index.html`

JSR 51: New I/O APIs for the Java platform (includes java.util.regex) · `jcp.org/en/jsr/detail?id=51`

15 NIO buffers and channels

Java 1.4 · JSR 51 · java.nio

Buffers, channels, selectors

The original `java.io` model is a *stream*: you read or write one byte (or char) at a time, and every read blocks the calling thread until data arrives. That is simple, but a server that handles ten thousand connections then needs ten thousand threads, most of them sleeping. NIO ("new I/O", 1.4) added a block-oriented model: data moves in `ByteBuffer`s, chunks of memory with a position and a limit, through `Channel`s (`FileChannel`, `SocketChannel`) that can be switched to non-blocking mode. A `Selector` lets one thread ask "which of these channels has data ready?" and serve only those. It also made character encoding explicit through `Charset`.

```
ByteBuffer buf = ByteBuffer.allocate(8192);
try (FileChannel ch = FileChannel.open(path)) {
    while (ch.read(buf) != -1) {  // fill
        buf.flip();               // -> read mode
        process(buf);             // drain
        buf.clear();              // -> write mode
    }
}
```

USE IT WHEN High-throughput file copies (`transferTo`), memory-mapped files, binary protocols, and servers with thousands of mostly idle connections. For ordinary file reading use `Files.readString` and `Files.lines` (NIO.2, Java 7 and 11, chapter 3) instead; they are built on the same channels.

DO Learn the buffer state machine: write into the buffer, `flip()`, read out of it, then `clear()` (everything consumed) or `compact()` (keep the unread tail). Reuse direct buffers; allocating them is expensive and their memory is freed late.

DON'T Don't hand-roll a selector loop for a business application; use Netty or a framework built on it, or let virtual threads (Java 21) make the simple blocking model cheap again. And never forget `flip()`: a buffer you filled but did not flip reads as empty.

REMEMBER *Stream = a pipe of bytes. Buffer = a bucket you fill, flip, and empty.*

IN DEPTH

Anatomy of a buffer. A buffer has a *capacity* (its size, fixed), a *position* (where the next read or write happens) and a *limit* (where reading or writing must stop). After you write three bytes into an empty 8-byte buffer, position is 3 and limit is 8. `flip()` sets limit to the current position (3) and position to 0: now reading will return exactly the three bytes you wrote. `clear()` resets position to 0 and limit to capacity for the next fill (it does not erase anything). `remaining()` is `limit - position`. The quiz question on this card asks you to predict those numbers; if you can, you understand NIO buffers.

Heap versus direct buffers. `ByteBuffer.allocate` creates a buffer backed by a Java `byte[]` on the heap. `allocateDirect` creates one in native memory outside the heap, which the operating system can read and write without copying, so it is faster for I/O but slower to allocate and invisible to the usual heap monitoring. Frameworks pool direct buffers for this reason.

Channels and selectors. A `FileChannel` adds `transferTo` (copy to another channel, often as a zero-copy system call), `map` (map a file region into memory and read it as a buffer), and positional reads. A `SocketChannel` in non-blocking mode returns immediately with however much data is available, possibly none. A `Selector` registers many channels with the interest you have in each (readable, writable, acceptable) and blocks until at least one is ready, returning the set of ready keys. One thread looping over that set can serve thousands of connections. This is the *reactor* pattern, and it is what Netty, Jetty, Tomcat's NIO connector, Vert.x and every reactive framework are built on.

Charsets. Converting between bytes and characters needs an encoding, and before 1.4 Java mostly used the platform default silently. `Charset`, `CharsetEncoder` and `CharsetDecoder` made the choice explicit and incremental. The platform default remained a trap (a file written on Windows in Cp1252 read on Linux as UTF-8) until Java 18 made UTF-8 the default everywhere (JEP 400).

What came next. NIO shipped without a decent file-system API; Java 7's NIO.2 (JSR 203, chapter 3) added `Path`, `Files`, directory walking and `WatchService`. Java 21's virtual threads (chapter 10) changed the calculus again: a blocking read on a virtual thread parks the thread cheaply, so thread-per-connection code becomes viable at scale without a selector loop.

WHY IT EXISTS

Around 2000 the "C10K problem" (serving ten thousand concurrent connections on one machine) was the benchmark for server platforms, and Java's one-thread-per-connection streams could not do it. JSR 51 gave Java the non-blocking primitives that operating systems already exposed in C, so that Java servers could compete.

TRY IT YOURSELF

Watch the buffer state machine with real numbers, then memory-map a file and read it through a buffer without a single `read()` call.

```
jshell> var b = java.nio.ByteBuffer.allocate(8)
jshell> b.put((byte) 1).put((byte) 2); b.position() + " " + b.limit() + " " + b.remaining()   // 2 8 6
jshell> b.flip(); b.position() + " " + b.limit() + " " + b.remaining()                      // 0 2 2
jshell> b.get(); b.get(); b.remaining()                                                      // 0
jshell> b.clear(); b.remaining()                                                             // 8

jshell> var ch = java.nio.channels.FileChannel.open(java.nio.file.Path.of("/etc/hosts"))
jshell> var map = ch.map(java.nio.channels.FileChannel.MapMode.READ_ONLY, 0, ch.size())
jshell> java.nio.charset.StandardCharsets.UTF_8.decode(map).toString().lines().findFirst()
```

REFERENCES

java.nio package: buffers · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/nio/package-summary.html`

ByteBuffer API · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/nio/ByteBuffer.html`

java.nio.channels package: channels and selectors · `docs.oracle.com/en/java/javase/21/docs/api/java.base/java/nio/channels/package-summary.html`

JSR 51: New I/O APIs for the Java platform · `jcp.org/en/jsr/detail?id=51`

JEP 400: UTF-8 by default (Java 18) · `openjdk.org/jeps/400`

16 assert and java.util.logging

Java 1.4 · JSR 41, JSR 47

Check invariants; log with levels

Two diagnostic tools arrived together in 1.4. The `assert` statement, `assert condition : message`, throws an `AssertionError` when the condition is false, but only if the JVM was started with `-ea` (enable assertions). By default the checks are compiled into the class file and skipped at run time, so they cost nothing in production. `java.util.logging` gave the JDK a built-in logging framework: named `Logger`s arranged in a hierarchy, a `Level` for each message (`SEVERE`, `WARNING`, `INFO`, `CONFIG`, `FINE`, `FINER`, `FINEST`), `Handler`s that send records to the console or files, and `Formatter`s that lay them out.

```
assert balance >= 0 : "negative: " + balance;
// run with:  java -ea App   (off by default!)

private static final Logger LOG =
    Logger.getLogger(Billing.class.getName());
LOG.log(Level.WARNING, "retry {0}", attempt);
LOG.fine(() -> "st " + expensive()); // lazy (8)
```

USE IT WHEN `assert` for conditions that must be true if your own code is correct (invariants, postconditions, "this branch is unreachable"), during development and in tests. Logging for everything an operator needs to see when the program runs in production.

DON'T Don't validate method arguments or user input with `assert`: in production the check vanishes and bad data flows through. Use `Objects.requireNonNull` or throw `IllegalArgumentException`. Don't put side effects in an assertion (`assert list.add(x)` works in tests and silently does nothing in production), and don't build debug messages with string concatenation.

DO Run your tests with `-ea` (Maven Surefire and Gradle do so by default) so assertions actually execute somewhere, and log through a facade such as SLF4J with parameterised messages (`log.debug("user {} logged in", id)`) so the arguments are only formatted when that level is enabled.

REMEMBER *assert is for bugs you don't expect, off in prod. Exceptions are for failures you do expect, always on.*

IN DEPTH

Assertions versus exceptions. An exception reports a failure that can legitimately happen at run time: the file is missing, the network is down, the caller passed a negative amount. An assertion documents something that *cannot* happen unless the program itself is wrong: "at this point the list is sorted", "this switch covers every case". Because production builds run with assertions off, they must never be the only thing standing between bad input and your data. `-ea` can be scoped: `-ea:com.ballys.payments...` enables them for one package tree, `-esa` enables the JDK's own system assertions.

How java.util.logging is organised. Loggers are named with dots (`com.ballys.payments.Billing`) and form a tree; a logger without its own level inherits its parent's, so setting `com.ballys` to `FINE` turns on debug output for the whole company's code at once. Each log record goes to the logger's handlers and, by default, up the tree to the root logger's `ConsoleHandler`. Configuration lives in a `logging.properties` file selected with `-Djava.util.logging.config.file`. The levels map roughly to the SLF4J names you may know: `SEVERE` = error, `WARNING` = warn, `INFO` = info, `FINE` = debug, `FINER` / `FINEST` = trace.

Why most projects log through something else. Log4j (2001) existed before `java.util.logging` and was more flexible; Logback and Log4j 2 followed. SLF4J became the common *facade*: libraries log to SLF4J, the application chooses the implementation, and bridges (`jul-to-slf4j`) route the JDK's own `java.util.logging` output into it. Java 9 added `System.Logger`, a minimal facade inside the JDK so that the platform's modules can log without depending on `java.util.logging`. In practice: use SLF4J in applications, know `java.util.logging` exists because the JDK and some libraries speak it.

Parameterised messages. `LOG.fine("state " + expensive())` computes the string, including the expensive call, even when `FINE` is off and the message is thrown away. `LOG.log(Level.FINE, "state {0}", state)` or the Java 8 supplier form `LOG.fine(() -> "state " + expensive())` defer the work until the logger has decided to print. In busy code paths this is the difference between free debug logging and a measurable slowdown.

WHY IT EXISTS

Every sizeable project had written its own logging and its own debug-check macros; JSR 47 and JSR 41 standardised both so that the JDK, libraries and applications could share one mechanism. Assertions were deliberately made cheap to leave in the code (off by default) so developers would write them freely, as C programmers did with `assert.h`.

TRY IT YOURSELF

Run the program below twice, with and without `-ea`, to see the assertion appear and disappear. Then raise the logger's level in code and watch the `FINE` line show up.

```java
import java.util.logging.*;
public class Diag {
    public static void main(String[] a) {
        Logger log = Logger.getLogger("demo");
        log.info("starting");
        log.fine("hidden: FINE is below the default INFO level");
        log.setLevel(Level.FINE);
        for (Handler h : Logger.getLogger("").getHandlers()) h.setLevel(Level.FINE);
        log.fine(() -> "visible now, and only built when needed");
        int balance = -5;
        assert balance >= 0 : "negative balance " + balance;   // only with -ea
        System.out.println("reached the end");
    }
}
// $ java Diag.java        -> reached the end
// $ java -ea Diag.java    -> AssertionError: negative balance -5
```

REFERENCES

JLS section 14.10: the assert statement · docs.oracle.com/javase/specs/jls/se21/html/jls-14.html#jls-14.10

Programming with Assertions (Java SE guide) · docs.oracle.com/javase/8/docs/technotes/guides/language/assert.html

Java Logging Overview (java.util.logging) · docs.oracle.com/en/java/javase/21/core/java-logging-overview.html

System.Logger API (Java 9 facade) · docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/System.Logger.html

JSR 41: a simple assertion facility · jcp.org/en/jsr/detail?id=41

JSR 47: logging API specification · jcp.org/en/jsr/detail?id=47

Dos and don'ts, Java 1.0–1.4 in one table

#	FEATURE	DO	DON'T
1	**Classes and inheritance**	Make classes `final` unless you designed them to be extended, make every field `private`, and override `equals` and `hashCode` together or not at all.	Don't inherit just to reuse code. `java.util.Stack` extends `Vector` to reuse its storage, so every `Vector` method became part of `Stack`'s public API, including `insertElementAt`, which lets a caller insert into the middle of a "stack". It can never be removed without breaking programs. Put the reused class in a field instead (composition).
2	**Interfaces**	Program to the interface: `List<String> xs = new ArrayList<>()`. Keep interfaces small; one or two methods is normal, not a smell.	Don't use an interface as a bag of constants (`interface Consts { int MAX = 9; }`): it leaks the constants into every implementing class's namespace. And don't add a method to an interface you have published: every existing implementor stops compiling (default methods in Java 8 exist precisely to soften this).
3	**Exceptions**	When you wrap a low-level exception in your own, always pass the original as the *cause* (second constructor argument), and catch the most specific type you can actually handle, nothing broader.	Don't write `catch (Exception e) {}` and carry on (the failure vanishes), don't both log and rethrow the same exception (it appears twice in the logs), and never `return` from inside `finally`: it silently throws away any exception that was in flight.
4	**JVM and bytecode**	Spend ten minutes with `javap -c` on a small class of your own (steps below). Seeing that a string `+` becomes a builder call, that an `int` in a `List` becomes `Integer.valueOf`, and that `synchronized` becomes `monitorenter` / `monitorexit` instructions makes those three topics concrete for good.	Don't assume "run anywhere" covers behaviour as well as bytecode: file path separators, line endings, the default character set (UTF-8 everywhere only since Java 18) and the locale still differ between platforms.
5	**Garbage collection**	Remove references from long-lived structures (caches, static registries, listener lists) when you no longer need the object, and set the heap size explicitly in containers (`-Xmx` or `-XX:MaxRAMPercentage`).	Don't call `System.gc()` from application code (it forces an expensive full collection and the JVM knows better when to run), and don't rely on `finalize()` to release resources (deprecated for removal since Java 18, JEP 421): use try-with-resources or `java.lang.ref.Cleaner`.
6	**HotSpot JIT**	Benchmark with JMH (the OpenJDK micro-benchmark harness) and let the JVM warm up before you measure. Keep hot methods small so the JIT can inline them (the default limit is about 35 bytes of bytecode for an ordinary method, more for frequently called ones).	Don't hand-optimise Java the way you would C: manual inlining, caching `array.length` in a local, avoiding getters "for speed". HotSpot does all of that itself, and your version only hides the intent from the next reader.
7	**Threads and synchronized**	Guard every access to shared mutable state, reads included, with the same lock, or make the state immutable so there is nothing to guard. Prefer a private lock object (`private final Object lock = new Object()`) over `synchronized (this)`, so outside code cannot interfere with your locking.	Don't call `run()` when you mean `start()` (it runs the body on your own thread, no concurrency at all), don't synchronize on `String` literals or boxed numbers (they are shared across the whole JVM, so unrelated code may lock the same object), and don't touch `Thread.stop`: it has thrown `UnsupportedOperationException` since Java 20.

#	FEATURE	DO	DON'T
8	**Inner and anonymous classes**	Declare nested classes `static` unless they genuinely need the enclosing instance. An inner class keeps a hidden reference to its outer object, which keeps that object alive and drags it into serialization.	Don't hand an inner-class or anonymous-class instance to something long-lived (a cache, a static registry, a listener you never remove): through its hidden `Outer.this` it keeps the whole outer object, and everything that object references, from being garbage collected. This is one of the classic Java memory leaks.
9	**Reflection and JavaBeans**	Cache `Method` and `Field` objects once you have looked them up (the lookup is slow, the call is not), and prefer `MethodHandle` (Java 7) or code generation at build time when performance matters.	Don't `setAccessible(true)` your way into JDK internals (`sun.misc.*`, private fields of `java.lang` classes). Since Java 16 the module system blocks it at run time (JEP 396, JEP 403) unless the user adds `--add-opens` flags, which is exactly what you want to avoid asking them to do.
10	**Serialization**	If you must use it: declare `serialVersionUID` explicitly (otherwise the compiler derives one from the class shape and any edit breaks old streams), mark passwords and connections `transient` so they are not written, and install a deserialization filter (`-Djdk.serialFilter=com.acme.*;!*`) that allows only your own classes.	Never deserialize bytes that arrived from a network or a user without a filter, and don't add `implements Serializable` "in case it is needed": it turns your private fields into a public file format you must support forever.
11	**Collections framework**	Declare variables, fields, parameters and return types as the interface (`List`, not `ArrayList`), and make `equals` and `hashCode` correct and consistent on anything that goes into a `HashSet` or is used as a `HashMap` key.	Don't add to or remove from a collection while iterating over it with for-each (you get `ConcurrentModificationException`; use an `Iterator` or `removeIf`), don't use a mutable object as a map key or set element, and don't reach for `LinkedList` expecting it to be faster: `ArrayList` wins almost every real benchmark.
12	**Iterator, Comparator, utilities**	Implement `Comparable` only for the one obvious order of a type, and keep it consistent with `equals` (if `compareTo` returns 0, `equals` should be true); everything else is a `Comparator`. Return `Collections.unmodifiableList(inner)` from getters, never the field itself.	Don't compute `a.x - b.x` in a comparator (it overflows for large values and gives the wrong sign; use `Integer.compare`), and don't believe `Collections.synchronizedMap` makes a check-then-put sequence atomic: it locks each call, not the pair.
13	**Vector, Hashtable, Enumeration**	Replace on sight: `Vector` → `ArrayList`, `Hashtable` → `HashMap`, or `ConcurrentHashMap` if threads share it, `Stack` → `ArrayDeque`, `Enumeration` → `Iterator` (`Collections.list(e)` or `e.asIterator()` since Java 9 convert).	Don't choose `Vector` or `Hashtable` "because they are thread-safe": a lock per method call does not make `if (!v.contains(x)) v.add(x)` atomic, since another thread can add between your two calls. `java.util.concurrent` (chapter 2) solves the actual problem with `putIfAbsent` and friends.
14	**Regular expressions**	Compile once into a `static final Pattern`, name your groups (`(?<year>\d{4})`, since Java 7) so `m.group("year")` reads better than `m.group(1)`, and anchor with `^` and `$` or use `matches()` when you mean the whole string.	Don't call `String.matches`, `split` or `replaceAll` inside a hot loop (each call recompiles the pattern), and don't run nested quantifiers such as `(a+)+` on input you do not control: on a non-matching string the engine tries exponentially many paths, and a few dozen characters can freeze a thread for minutes (a denial-of-service known as catastrophic backtracking).

#	FEATURE	DO	DON'T
15	**NIO buffers and channels**	Learn the buffer state machine: write into the buffer, `flip()`, read out of it, then `clear()` (everything consumed) or `compact()` (keep the unread tail). Reuse direct buffers; allocating them is expensive and their memory is freed late.	Don't hand-roll a selector loop for a business application; use Netty or a framework built on it, or let virtual threads (Java 21) make the simple blocking model cheap again. And never forget `flip()`: a buffer you filled but did not flip reads as empty.
16	**assert and java.util.logging**	Run your tests with `-ea` (Maven Surefire and Gradle do so by default) so assertions actually execute somewhere, and log through a facade such as SLF4J with parameterised messages (`log.debug("user {} logged in", id)`) so the arguments are only formatted when that level is enabled.	Don't validate method arguments or user input with `assert`: in production the check vanishes and bad data flows through. Use `Objects.requireNonNull` or throw `IllegalArgumentException`. Don't put side effects in an assertion (`assert list.add(x)` works in tests and silently does nothing in production), and don't build debug messages with string concatenation.

Don't mix these up

Checked vs. unchecked exception

Checked extends **Exception** and must appear in `throws` or be caught. Unchecked extends **RuntimeException** (or Error) and may be ignored by the compiler, not by you.

Interface vs. abstract class

An interface is a **contract**; a class may implement many. An abstract class is a **partial class** with state and constructors; you get exactly one.

== vs. equals()

`==` compares **references** (same object). `equals()` compares **values** as the class defines them. Strings, boxed numbers, collections: always equals().

Inner vs. static nested class

An inner class holds a hidden **Outer.this** and cannot exist without an instance. A static nested class is **just a namespace**. Default to static.

Comparable vs. Comparator

`Comparable` is the type's **own** order (`compareTo`, one per class). `Comparator` is an order supplied **from outside**, as many as you like.

Vector vs. ArrayList

Same idea; `Vector` locks on **every call** and is still unsafe for check-then-act. `ArrayList` is unsynchronised and faster; use j.u.c when sharing.

start() vs. run()

`start()` asks the JVM for a **new thread**, which then calls run(). Calling `run()` yourself runs the body on the **current** thread, synchronously.

final vs. finally vs. finalize()

`final`: no reassignment, no override, no subclass. `finally`: runs on **every exit** from try. `finalize()`: a GC hook, deprecated, never use it.

Feature names and JEP numbers from openjdk.org. Explanations, examples and advice are our own wording, for personal study. Knowledge graph, cheat sheet, quiz and practice exercises for this chapter: see the by-version folder and practice/.